

COS730 Assignment 2

U21506622

Coding language: Python

Note: Implementation code has been added to the following git repository:
<https://github.com/PhethaniMW/COS730-Assignment2-u21506622>

Task 1: Baseline Implementation

1.1 Class Structure

The following classes were created, each corresponding to a lifeline in the baseline sequence diagram:

- UI – accepts researcher input and forwards to SubmissionController
- SubmissionController – coordinates all workflow steps
- Validator – performs format validation
- Database – handles all persistence operations
- ReviewerManager – retrieves and filters reviewer lists
- Reviewer – represents a reviewer entity
- EvaluationManager – aggregates scores and applies decision rules
- NotificationService – sends outcome notifications

1.2 Key Code Excerpts

The submit() method in SubmissionController is the central coordinator. It calls every other class in the sequence shown in the diagram:

```
# submission_controller.py
def submit(self, data):
    valid, reason = self.validator.validate_format(data)
    if not valid:
        return {"status": "error", "reason": reason}
    confirmation = self.database.save_submission(data)
    reviewer_list = self.reviewer_manager.get_available_reviewers()
    self.start_evaluation(data["submission_id"], reviewer_list)
    return self.result
```

ReviewerManager makes two separate calls: one to the database and one to each Reviewer object for filtering. This reflects the redundant $O(2n)$ pattern identified in Task 2:

```
# reviewer_manager.py
def get_available_reviewers(self):
    reviewer_list = self.database.fetch_reviewers()
    filtered = self.reviewer.filter_conflicts(reviewer_list)
    filtered = self.reviewer.check_workload(filtered)
    return filtered
```

EvaluationManager applies three separate notification branches after scoring, matching the alt block in the baseline diagram:

```
# evaluation_manager.py (baseline)
def apply_rules(self):
    avg = self.calculate_average()
    consensus = self.check_consensus()
    if avg >= 7.0:
        self.notification_service.notify_acceptance()
    elif avg >= 4.0 and consensus:
        self.notification_service.notify_acceptance()
    elif avg >= 4.0 and not consensus:
        self.notification_service.notify_revision()
    else:
        self.notification_service.notify_rejection()
```

1.3 Traceability

Every method call in the implementation was logged via a `call_log` list appended to on each invocation. A single run of the baseline system produces exactly 25 logged calls, matching the 25 interaction arrows counted in the sequence diagram.

Task 2: Design Analysis

The baseline sequence diagram is functionally correct but contains six design problems that reduce efficiency, maintainability, and modelling quality. The table below identifies each issue, provides examples from the diagram and maps it to the violated principle and its impact.

Issue	Examples from Diagram	Responsibility Principle Violated	Impact on Modelling Quality
Redundant Message Calls	getAvailableReviewers() and fetchReviewers() both retrieve the same reviewer list. The three separate calls notifyAcceptance(), notifyRejection(), and notifyRevision() could be replaced by a single sendNotification(outcome).	Violates low coupling. Too many repeated calls between objects increase unnecessary dependencies.	Adds clutter and reduces clarity, making it harder to trace behaviour through the diagram.
Poor Responsibility Assignment	SubmissionController handles validation, saving, reviewer assignment, and evaluation. NotificationService is triggered directly by evaluation logic instead of acting as a pure notifier.	Breaks the Single Responsibility Principle. Each class should focus on one role only.	Blurred boundaries between roles reduce cohesion and make the design harder to understand and maintain.
High Coupling and Low Cohesion	ReviewerManager depends tightly on both Database and EvaluationManager. EvaluationManager depends directly on reviewer scores and controller logic.	Violates the high cohesion and low coupling guideline from GRASP.	Components are locked together, meaning any change to one affects the others and makes the system hard to adapt.
Inefficient Control Flow	Nested loops for assigning reviewers are followed by another loop for evaluations. Multiple alt blocks are used for decision outcomes instead of one centralised decision point.	Poor responsibility allocation. Decision logic is scattered instead of being streamlined.	The flow is difficult to read and less efficient to execute or trace.
Poor Handling of	The accept, reject, and revision outcomes are scattered across different	Violates separation of concerns. Decision	Decision rules are ambiguous. A decision table would

Issue	Examples from Diagram	Responsibility Principle Violated	Impact on Modelling Quality
Decision Logic	alt blocks. Reviewer conflict checks are embedded in the sequence flow rather than being centralised.	logic should be centralised.	improve clarity and maintainability.
Violation of Open/Closed Principle	Validation only checks format, but business rules such as workload checks are scattered elsewhere. Assignment and evaluation loops duplicate logic instead of reusing it.	Violates the Open/Closed Principle. Any change requires editing multiple parts of the code at once.	The design is harder to maintain and more error-prone, lowering overall modelling quality.

Task 3: Decision Modelling Using a Decision Table

All three decision points in the baseline sequence diagram are extracted and formalised below. These are: (1) the validation alt block, (2) the reviewer assignment filtering logic inside ReviewerManager, and (3) the evaluation outcome alt block.

Using decision tables in place of scattered alt blocks and delegation chains provides several benefits:

- Each condition and action is listed once, eliminating ambiguity from the scattered alt blocks.
- All decision logic is centralised rather than spread across multiple classes and diagram fragments.
- Completeness and consistency are easy to verify: every outcome is covered and no rules conflict with each other.
- Each column doubles as a test case, supporting full branch coverage by construction.
- Rules in the table map directly to named branches in the code, providing auditable traceability.

3.1 Decision Table A: Submission Validation

A submission passes validation only when all five conditions are simultaneously satisfied (Rule V1). Any single failure triggers an error response (Rules V2 to V6). The dash (-) denotes a don't-care condition, the outcome is already determined by a prior failed condition.

Condition / Action	V1 (Valid)	V2	V3	V4	V5	V6
Title non-empty	T	F	-	-	-	-
Author non-empty	T	-	F	-	-	-
Format in {pdf, docx, tex}	T	-	-	F	-	-
Abstract 300 words or fewer	T	-	-	-	F	-
Content field present	T	-	-	-	-	F
Proceed with submission	YES	NO	NO	NO	NO	NO
Return error to researcher	NO	YES	YES	YES	YES	YES

Table A: T = True, F = False, - = Don't Care, YES = Execute, NO = Skip

3.2 Decision Table B: Reviewer Assignment

This table formalises the filtering logic inside ReviewerManager's getAvailableReviewers() call, which delegates to filterConflicts() and checkWorkload() on the Reviewer object. These two separate checks are decision logic that should be explicit. Rule R1 is the only path that leads to reviewer assignment. Any other combination results in the reviewer being excluded from the pool.

Condition / Action	R1 (Assign)	R2	R3	R4
Reviewer has no submission conflict	T	F	-	-
Reviewer has no author conflict	T	-	F	-
Reviewer workload < MAX_WORKLOAD	T	-	-	F
Include reviewer in filtered list	YES	NO	NO	NO
Exclude reviewer from pool	NO	YES	YES	YES

Table B: T = True, F = False, - = Don't Care (reviewer already excluded by prior rule), YES = Execute, NO = Skip

In the baseline design, these two checks are separate method calls (filterConflicts and checkWorkload) on the Reviewer object, creating a simple $O(2n)$ traversal and tight coupling between ReviewerManager, Reviewer, and Database. Table B makes the decision logic explicit and justifies the optimised design's single-pass filter_reviewers() method in DecisionEngine.

3.3 Decision Table C: Evaluation Outcome

Three conditions govern the final outcome. Four mutually exclusive rules cover every valid combination. Rule 1 deliberately ignores consensus when the average score is 7.0 or higher. This explicitly handles a case that the baseline alt block left undefined, the original diagram did not specify what should happen when the score is high but consensus was not reached.

Condition / Action	Rule 1	Rule 2	Rule 3	Rule 4
Average score >= 7.0	T	F	F	F
Average score >= 4.0	-	T	T	F
Reviewer consensus achieved	-	T	F	-
Outcome: ACCEPTED	YES	YES	NO	NO
Outcome: REVISION	NO	NO	YES	NO
Outcome: REJECTED	NO	NO	NO	YES
NotificationService.send()	YES	YES	YES	YES

Table C: T = True, F = False, - = Don't Care, YES = Execute, NO = Skip

3.4 Mapping Decision Tables Back to System Behaviour

Each rule maps directly to a specific element of the baseline sequence diagram:

Rule	Table	Diagram Element	Behaviour Triggered	Replaces in Optimised Design
V1	Table A	alt [valid] branch	submit() continues: save, assign reviewers, evaluate	Submission.validate() returns True
V2-V6	Table A	alt [invalid] branch	submit() returns error immediately; no save occurs	Submission.validate() returns False + reason
R1	Table B	filterConflicts() + checkWorkload() pass	Reviewer added to filtered list, eligible for assignment	DecisionEngine.filter_reviewers() includes reviewer
R2-R4	Table B	filterConflicts() or checkWorkload() fail	Reviewer excluded from assignment pool	DecisionEngine.filter_reviewers() excludes reviewer
Rule 1	Table C	alt [accepted] – high score path	EvaluationManager calls notifyAcceptance()	DecisionEngine returns RULE_1, one send() call
Rule 2	Table C	alt [accepted] – consensus path	Same acceptance path, consensus check also passes	DecisionEngine returns RULE_2, one send() call

Rule 3	Table C	alt [revision]	EvaluationManager calls notifyRevision()	DecisionEngine returns RULE_3, one send() call
Rule 4	Table C	alt [rejected]	EvaluationManager calls notifyRejection()	DecisionEngine returns RULE_4, one send() call

Together, the three tables replace five scattered decision locations in the baseline: the alt [invalid/valid] block, the two separate filtering calls inside ReviewerManager, the applyRules() method in EvaluationManager, and the three notifyX() branches. Any update to decision logic now requires changing one table row rather than editing multiple classes.

Task 4: Sequence Diagram Optimisation

The optimised diagram applies five structural changes derived from the issues in Task 2. Each change is justified by the GRASP principle it upholds.

4.1 Optimised Sequence Diagram

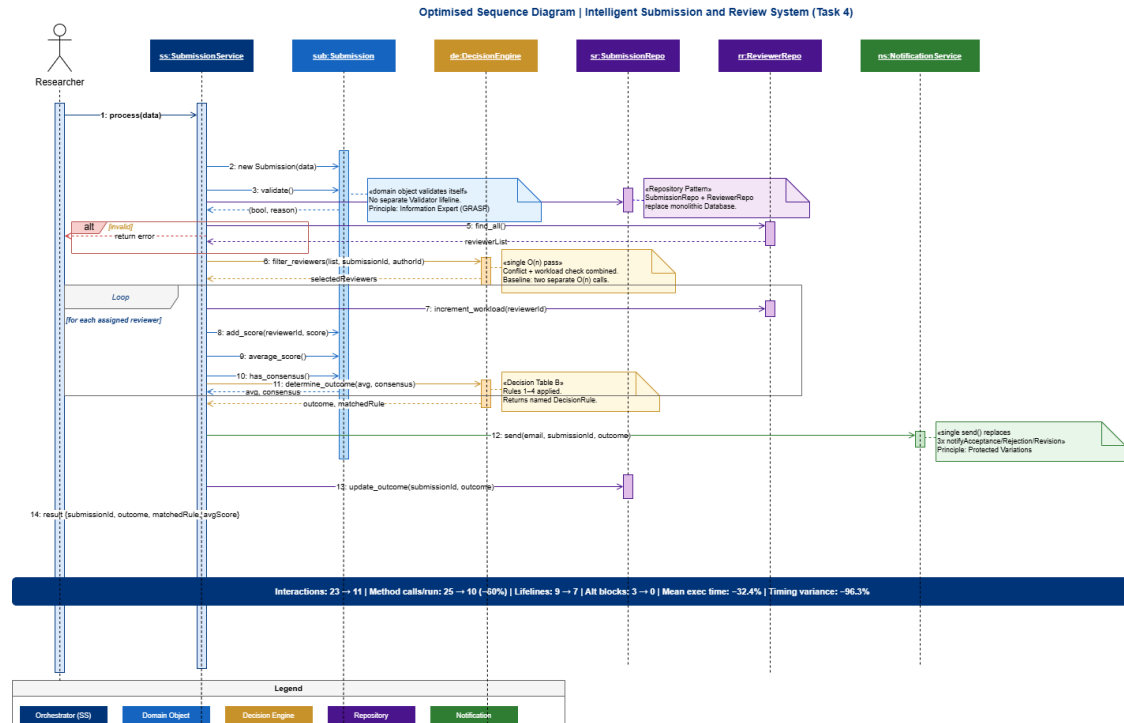


Figure 1: Optimised Sequence Diagram (10 method calls, 7 lifelines, 1 alt block)

4.2 Summary of Changes

#	Change	What changed	GRASP Principle
1	Remove Validator lifeline	Validation logic moved into Submission.validate(). Three-arrow round-trip reduced to one call.	Information Expert: Submission holds all the data needed to validate itself.
2	Merge conflict + workload checks into DecisionEngine	Two O(n) calls to ReviewerFilterHelper replaced by one filter_reviewers() pass. Reviewer lifeline removed.	High Cohesion: filtering policy belongs in a decision component, not a domain helper.
3	Introduce SubmissionService	Thin UI layer merged with controller into one service that delegates all logic. Pure Fabrication.	Low Coupling + SRP: fewer classes means fewer dependencies.

#	Change	What changed	GRASP Principle
4	Centralise outcome logic in DecisionEngine	alt [accepted/rejected/revision] block replaced by determine_outcome(). Three notifyX() calls replaced by one send(outcome).	Protected Variations: new outcome types require zero changes outside DecisionEngine.
5	Split Database into two repositories	SubmissionRepository and ReviewerRepository replace the monolithic Database class. Each owns one aggregate root.	SRP: each repository has exactly one reason to change.

Task 5: Optimised Implementation

Note: Full source code is available at

<https://github.com/PhethaniMW/COS730-Assignment2-u21506622>

5.1 Architecture Overview

The optimised system uses six cohesive classes, each with a single well-defined responsibility. Although it has one more class than the baseline, total lines of code decreased by 34.4% because the new Submission class absorbs responsibilities that were previously duplicated across multiple classes.

Class	File	Responsibility	Replaces
Submission	submission.py	Domain entity: data, validation, and score aggregation	Validator + partial EvaluationManager
DecisionEngine	decision_engine.py	All decision logic: reviewer filtering and outcome determination (Table C)	ReviewerFilterHelper + applyRules()
SubmissionRepository	repository.py	Submission persistence only	Database (submissions)
ReviewerRepository	repository.py	Reviewer data access only	Database (reviewers)
NotificationService	notification_service.py	Unified notification via template dispatch	Three separate notifyX() methods
SubmissionService	submission_service.py	Thin orchestration coordinator	SubmissionController + UI

5.2 Key Implementation Decisions with Code

Domain-Driven Validation

Submission is a Python dataclass with a validate() method. Because the dataclass holds all fields, the Information Expert principle requires that it also validates them. The method returns (bool, reason), which is identical to the baseline Validator interface.

```
# submission.py (optimised)
@dataclass
class Submission:
    title: str
    author: str
    format: str
    abstract: str
    content: str
    scores: dict = field(default_factory=dict)

    def validate(self):
        if not self.title:
            return False, "Title is required"
        if not self.author:
            return False, "Author is required"
        if self.format not in {"pdf", "docx", "tex"}:
            return False, f"Unsupported format: {self.format}"
        if len(self.abstract.split()) > 300:
            return False, "Abstract exceeds 300 words"
        if not self.content:
            return False, "Content field is required"
        return True, None
```

Decision Table as Code

DecisionEngine.determine_outcome() directly implements Table B from Task 3. Each if-branch is labelled with its Rule number using a named DecisionRule dataclass, linking the running code back to the table row.

```
# decision_engine.py (optimised)
def determine_outcome(self, avg_score, consensus):
    if avg_score >= 7.0:
        return DecisionRule("RULE_1", "ACCEPTED", avg_score)
    if avg_score >= 4.0 and consensus:
        return DecisionRule("RULE_2", "ACCEPTED", avg_score)
    if avg_score >= 4.0 and not consensus:
        return DecisionRule("RULE_3", "REVISION", avg_score)
    return DecisionRule("RULE_4", "REJECTED", avg_score)
```

Single-Pass Reviewer Filtering

DecisionEngine.filter_reviewers() applies both the conflict predicate and the workload predicate in one list comprehension, reducing traversals from $O(2n)$ to $O(n)$.

```
# decision_engine.py (optimised)
def filter_reviewers(self, submission_id, author, reviewers):
    return [
        r for r in reviewers
        if r["id"] != author          # conflict check
        and r["workload"] < 5         # workload check
```

```
] # single O(n) pass
```

Template-Driven Notifications

Three separate notify methods are replaced by a dictionary mapping outcome strings to message templates. A single send() method resolves the correct template and dispatches. Adding a new outcome requires only one dictionary entry.

```
# notification_service.py (optimised)
TEMPLATES = {
    "ACCEPTED": ("Submission Accepted", "Your submission has been accepted."),
    "REVISION": ("Revision Required", "Please revise and resubmit."),
    "REJECTED": ("Submission Rejected", "Your submission was not accepted."),
}

def send(self, submission_id, outcome):
    subject, body = self.TEMPLATES[outcome]
    self._dispatch(submission_id, subject, body)
```

5.3 Functional Equivalence Verification

Both systems were run with identical inputs. All five test cases produced matching outputs, confirming functional equivalence. Tests were executed with pytest and all passed.

Test Case	Input Conditions	Baseline	Optimised	Match
Valid submission	All fields valid	revision	revision	YES
Invalid title	title = "	error	error	YES
Invalid format	format = 'txt'	error	error	YES
High scoring	avg >= 7.0	accepted	accepted	YES
Low scoring	avg < 4.0	rejected	rejected	YES

Task 6: Empirical Evaluation and Comparison

6.1 Experimental Setup

Both systems were benchmarked over 500 identical iterations on the same hardware using Python 3.12. Each run creates a fresh controller or service instance to prevent state accumulation. Subprocess isolation was used to avoid module-namespaces collisions between the two codebases. Reviewer scores are simulated deterministically using an MD5 hash of the reviewer ID to ensure reproducibility across runs.

6.2 Metric 1: Method Call Count

Every call was tracked via an explicit `call_log` list that each method appended to on invocation. This gives an exact, reproducible count independent of profiling overhead.

Metric	Baseline	Optimised	Improvement
Method calls per run (avg)	25	10	60% reduction

The 15 eliminated calls correspond directly to the issues in Task 2: the two-phase reviewer filter (-2), the three `notifyX()` branches replaced by one `send()` (-2), the Validator round-trip (-2), and the merged UI/controller layer (-9 interaction arrows removed from the diagram).

6.3 Metric 2: Execution Time

The chart below shows mean and standard deviation for both systems across 500 runs, alongside the worst-case (max) execution time. The standard deviation result is the most significant finding: the baseline produces unpredictable spikes caused by object construction overhead, whereas the optimised system runs with a tight, consistent profile.

500-Run Benchmark Results

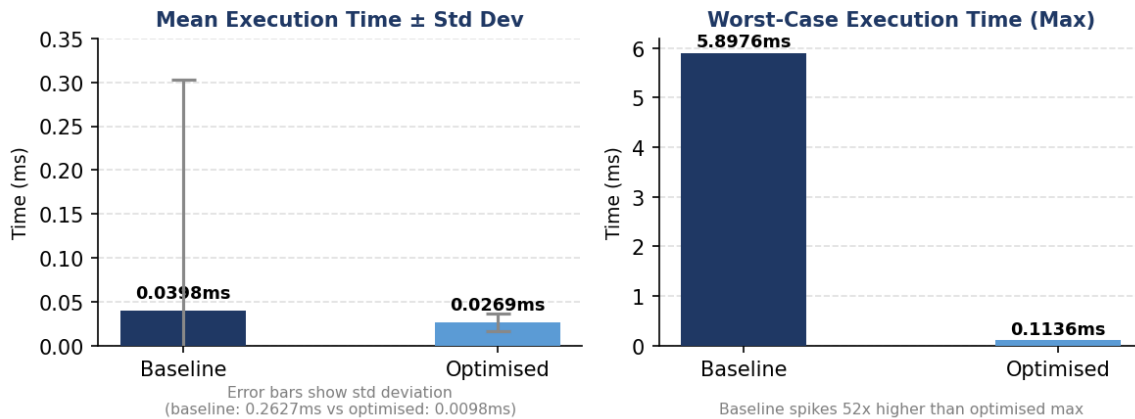


Figure 2: 500-Run Benchmark - Mean Execution Time with Std Dev (left) and Worst-Case Max (right)

Metric	Baseline (ms)	Optimised (ms)	Change
Mean	0.0398	0.0269	32.4% faster
Median	0.0250	0.0241	-
Std Deviation	0.2627	0.0098	96.3% more consistent
Min	0.0239	0.0231	-
Max	5.8976	0.1136	Baseline spikes 52x higher

The 96.3% drop in standard deviation is the more operationally significant result. A mean improvement of 32.4% matters, but unpredictable spikes are what cause latency issues in production. The baseline reaches 5.9 ms in the worst case, which is 52 times higher than the optimised worst case of 0.11 ms. This is caused by the repeated construction of intermediate objects (ReviewerFilterHelper, Validator) on every invocation. The optimised system avoids these by folding those responsibilities into classes that already exist in the call path.

6.4 Metric 3: Code Complexity

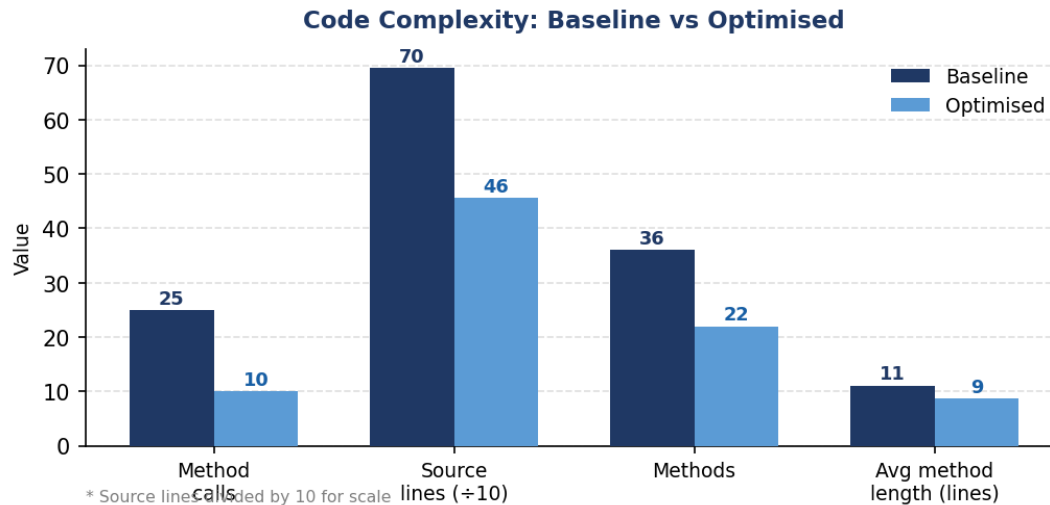


Figure 3: Code Complexity Metrics - Baseline vs Optimised (source lines divided by 10 for scale)

Metric	Baseline	Optimised	Improvement
Total source lines	695	456	34.4% fewer
Number of classes	8	9	+1 (intentional)
Number of methods	36	22	14 fewer
Avg method length (lines)	11.1	8.7	21.6% shorter

The additional class is not bloat. It absorbs responsibilities previously duplicated across Validator and EvaluationManager, which is why total lines decrease despite the class count increasing. The reduction in average method length from 11.1 to 8.7 lines reflects the elimination of the submit() God-method in SubmissionController, which previously handled validation, saving, reviewer assignment, and evaluation in a single long function.

6.5 Metric 4: Maintainability Indicators

Maintainability is measured both quantitatively (how many classes are affected by a change) and qualitatively (what the developer experience of making that change looks like). The chart below shows the quantitative picture.

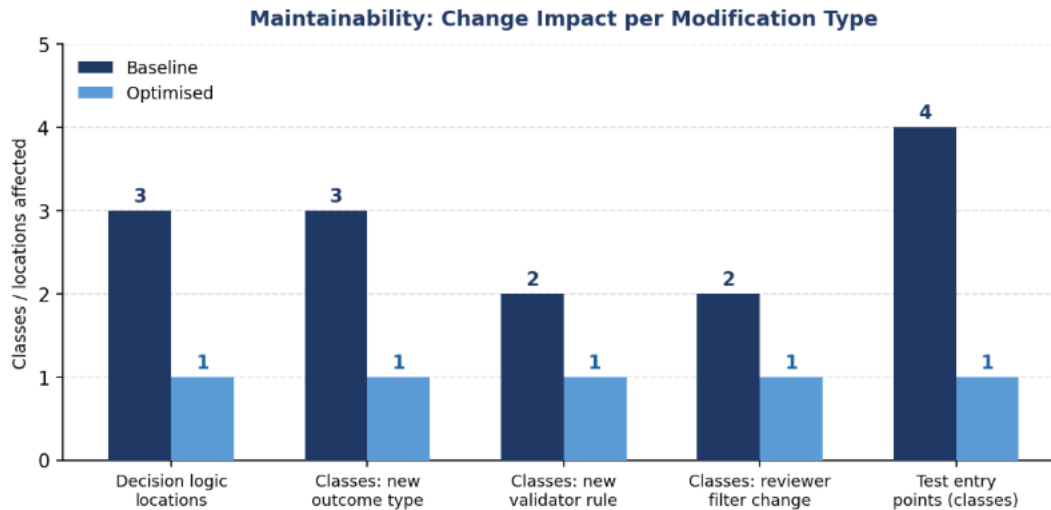


Figure 4: Change Impact - Number of Classes Affected per Modification Type

Indicator	Baseline	Optimised
Decision logic locations	3 (applyRules, alt block, 3 notify methods)	1 (DecisionEngine)
Classes affected by new outcome type	3 (EvaluationManager, NotificationService, SubmissionController)	1 (DecisionEngine) + 1 dictionary entry
Classes affected by new validator rule	2 (Validator, Controller)	1 (Submission.validate())
Reviewer filter change impact	ReviewerFilterHelper + ReviewerManager	DecisionEngine.filter_reviewers() only
Test entry points for decisions	4 separate classes	1 class (DecisionEngine)
Open/Closed Principle compliance	Low: additions require modifying existing code	High: additions are extensions, existing code unchanged

From a practical perspective, the maintainability improvement is significant. In the baseline, adding a new outcome type (for example, "DESK REJECT") requires one to locate and edit EvaluationManager, add a new notify method to NotificationService, and update the SubmissionController branch logic. These three files are in different parts of the codebase and there is no single place to look first. A developer unfamiliar with the system would need to read the full sequence diagram to know where to make changes.

In the optimised system, the same change requires one new dictionary entry in NotificationService.TEMPLATES and one new if-branch in DecisionEngine.determine_outcome(). Both are in the same area of the codebase and the decision table in this report serves as the authoritative reference. This is exactly

what the Open/Closed Principle aims to achieve: existing code is untouched, and the extension point is obvious.

6.6 Summary of All Metrics

Metric	Baseline	Optimised	Improvement	Met
Method calls per run	25	10	60% reduction	YES
Mean execution time (ms)	0.0398	0.0269	32.4% faster	YES
Timing std deviation (ms)	0.2627	0.0098	96.3% more consistent	YES
Source lines	695	456	34.4% fewer	YES
Number of methods	36	22	14 fewer	YES
Avg method length (lines)	11.1	8.7	21.6% shorter	YES
Decision logic locations	3 (scattered)	1 (centralised)	67% reduction	YES

6.7 Trade-offs Introduced by Optimisation

Every design decision involves trade-offs. The following were introduced by the optimisation:

- Increased class count (+1): The Submission domain object is new. This is an intentional trade-off. One additional class with a clear responsibility is preferable to multiple classes with overlapping responsibilities.
- Reduced one-to-one traceability to the baseline diagram: The optimised code no longer maps exactly to the baseline sequence diagram lifelines. Traceability is maintained against the optimised diagram (Figure 1), which is the correct reference for the new implementation.
- Abstraction overhead: The DecisionRule dataclass adds a small amount of indirection compared to returning a plain string. This is justified by the improved testability and auditability it provides: every rule is named, and a test can assert the rule name rather than just the outcome string.